IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Manuscript prepared for author review, October 19, 2026.

Digital Object Identifier

Mixed-Precision Robot Policy Inference: A Retrospective Evaluation of Numerical Fidelity, Latency, and Resource Costs

AUTHOR INFORMATION TO BE COMPLETED BEFORE SUBMISSION

Affiliation, author order, and correspondence details require author confirmation.

Corresponding author information pending confirmation.

Research manuscript based on a retrospective analysis of existing inference artifacts. Author metadata and biographies must be completed before submission.

ABSTRACT Robot policy deployment requires numerical fidelity, latency, and resource measurements with compatible boundaries. We retrospectively analyze saved inference artifacts, expanding a Jetson Thor archive through read-only inspection of online robot computers. Hash-linked 30-step outputs and a separately identifier-paired 115-input, 60-step cohort reveal both favorable and adverse mixed-precision configurations. On one 32-input sequence, mixed configurations reduced median recorded latency from 109.87 ms to 81.76 and 67.70 ms, while minimum input cosine fell to 0.928032 and 0.965292. Additional 50-step native-action replays permit nominal forward-kinematic analysis. In a four-camera, 26-coordinate replay with explicitly matched noise, a TensorRT split configuration retained aggregate cosine 0.999999645 against the stored reference but produced a maximum left-flange discrepancy of 4.867 mm. Its historical prepared-call median was 209.31 ms versus 287.35 ms for PyTorch; the associated PyTorch allocation counter increased from 8.16 to 8.84 GiB and does not include all backend allocations. A separate 16-coordinate archive shows larger conditional kinematic differences and an unresolved fixture–preprocessing schema mismatch. Resource evidence further distinguishes a 14.36% increase in retained weight storage from synthetic cache-byte reductions and a single stream's allocator and GPU-rail observations. Reanalysis exposed a report-to-reference mismatch and repeated inputs that would otherwise distort interpretation. The results support preserving per-input and kinematic discrepancy alongside explicit timing and resource boundaries. They provide reproducible retrospective evidence without establishing closed-loop task equivalence or comparative energy savings.

INDEX TERMS Edge computing, mixed precision, post-training quantization, robot learning, vision-language-action models.

I. INTRODUCTION

ROBOT policies increasingly combine visual observations, language instructions, and proprioception within a single inference pipeline. Models such as OpenVLA and $\pi_{0.5}$ extend this approach across manipulation settings and robot embodiments [1], [2]. Their computational demands make inference efficiency a practical constraint for onboard deployment. The relevant engineering question is how an accelerated implementation changes the actions available to the controller under the resources actually retained on the device.

Post-training quantization (PTQ) provides one route to reducing arithmetic and data movement. However, the nominal precision of a matrix multiplication does not specify the behavior of a complete policy. A deployment can combine low-precision projections, higher-precision normalization, calibration scales, runtime quantization, fused kernels, and cached weights. Implementation changes can also alter output values before any reduction in nominal precision occurs. Consequently, a reported speed ratio needs an explicit reference implementation and measurement boundary. We consider bfloat16 (BF16), eight-bit floating point (FP8), and NVIDIA four-bit floating point (NVFP4) configurations.

Robot-specific quantization methods increasingly account for decision-critical states, action sensitivity, and denoising dynamics [3]–[6]. Their results motivate an engineering question: whether the evidence retained during deployment preserves those distinctions when reduced to aggregate agree-

ment and a speed ratio.

An archived deployment offers both useful measurements and interpretive hazards. Files can reuse observations, concatenate state sweeps with camera records, or repeat a small fixture to extend a benchmark. A validation summary may evaluate fidelity on many inputs while timing only one. A quantized sidecar may coexist with the original packed weights. Treating these records as interchangeable can inflate the apparent sample count or conflate storage, memory, and task performance.

This study investigates those issues through a secondary analysis of existing $\pi_{0.5}$ deployment artifacts associated with Jetson Thor, complemented by a separate LingBot-VA video-action deployment archive [7]. A fleet-wide read-only inventory adds two 50-step checkpoint families with native joint targets, enabling nominal kinematic discrepancy to be evaluated alongside output similarity. The primary evidence consists of paired action outputs and latency arrays whose input identities use the 256-bit Secure Hash Algorithm (SHA-256). Additional evidence includes replay fixtures, calibration manifests, acceptance summaries, source files, and serialized weight-file metadata. All new computations analyze saved data offline; no robot was commanded and no new inference experiment was performed on the remote system.

Three research questions organize the analysis. First, how much output discrepancy accompanies a change between retained BF16 implementations? Second, how do component ablations and mixed-precision configurations differ in aggregate, input-level, and nominal end-effector fidelity? Third, how do storage, runtime allocation, cache precision, and complete-cycle timing differ when their measurement boundaries are traced?

The contribution is an empirical deployment evaluation rather than a new quantization algorithm. It provides a hash-linked reanalysis that separates real observations from constructed fixtures, a comparison of numerical reference choices and component-level failure patterns, a native-action kinematic analysis across two additional checkpoint families, and an audit of timing and resource denominators. The supplied scripts, metric tables, and artifact inventory make each reported result traceable to its acquired evidence. Conclusions remain specific to the archived checkpoint families, task contexts, and available measurement conditions.

II. RELATED WORK

A. QUANTIZATION AND IMPLEMENTATION COST

General PTQ methods differ in what they preserve and where their costs occur. GPTQ uses approximate second-order information for weight quantization [8]. Activation-aware Weight Quantization (AWQ) uses activation statistics to protect salient weight channels through scaling [9]. SmoothQuant redistributes activation outliers through an equivalent transformation to facilitate low-precision computation [10]. These methods provide relevant tools, but language-model evaluations do not establish robot task performance.

Low-bit formats also require a supported implementation. Official Triton examples distinguish block-scaled arithmetic from its packing, scaling, and kernel organization [11]. NVIDIA documentation describes different quantized representations and their associated scaling rules [12]. These sources inform terminology only: the archived deployment uses its own Triton-based paths, and cohorts A–D use these paths. Additional cohorts E/F retain PyTorch and TensorRT comparisons with different precision scopes. The separate LingBot-VA archive uses a TensorRT FP8 core with paged attention. We therefore report a configuration's observed behavior rather than predicting its complete-policy latency from nominal bit width.

B. ACTION-SENSITIVE ROBOT POLICY QUANTIZATION

SQIL links quantization to decision-critical states through saliency-aware training and evaluates policy outcomes [3]. QVLA measures action sensitivity to guide channel-level precision allocation [4]. HoloQ-VLA combines complementary rotations with denoising-step scaling for uniform low-bit quantization [5]. CHASE-VLA adapts action-expert activation scales using previously generated chunks and denoising-step information [6]. Its action-expert timing boundary differs from a complete policy call, illustrating why published speed ratios require compatible denominators.

Our analysis complements these methods by examining the retained evidence of a particular engineering deployment. We neither implement these methods as competing baselines nor claim higher task success. Their results should not be numerically ranked against our archived latency values because models, hardware, input shapes, and timing scopes differ. The question here concerns the information lost when deployment evidence is reduced to one aggregate similarity and one speed ratio.

LingBot-VA combines autoregressive video prediction and robot action generation with persistent context [7]. FlashInfer provides paged attention implementations for varied cache layouts and shapes [13]. These methods motivate a separate systems analysis of cache precision and full-cycle costs. Synthetic attention errors and timings do not establish action accuracy, and the acquired LingBot stream provides no matched whole-policy precision comparison.

C. MEASUREMENT AND UNCERTAINTY

MLPerf Inference illustrates the importance of specifying workloads and measurement rules when comparing inference systems [14]. Kalibera and Jones explain why repeated executions and within-execution iterations represent different sources of variation [15]. Work on statistical reporting in reinforcement learning similarly cautions against conclusions based on inadequately characterized repetition [16]. Our archive does not provide the independently sampled runs needed for population-level significance claims. We use descriptive effects and deterministic deletion sensitivity, while preserving the distinction between observations, repeated timings, and recordings.

III. MATERIALS AND EVALUATION METHOD

A. ACQUISITION AND EVIDENCE LEVELS

The archive was acquired on October 19, 2026, from an existing research directory on a machine identified as kitt15-025. The device roster described the host as a Jetson AGX Thor Developer Kit. This identifies the acquisition host, without independently certifying the original execution device of every archived file. An existing trusted SSH host key and the hostname were checked before reading files. Acquisition used SFTP and an uncompressed, explicitly allowlisted file stream. No model execution, robot interface, configuration write, service change, or control operation was invoked. Ordinary authentication records and file-access metadata may result from reads.

Transfer manifests preserve selected-file sizes, modification times and local SHA-256 values. These identify the acquired evidence copy without proving an atomic historical snapshot; exact transfer counts and full digests are supplied in the supplement.

The fifth-round search recursively inspected accessible research/data roots across the online-roster union. A private ledger retains directory frontiers, authentication/permission errors, excluded trees and unread payloads. Container headers, closed MCAP summaries and Parquet footers characterize recorded data; selected existing arrays and source records were copied locally. This discovery inventory does not constitute a multisystem performance benchmark or count of robot trials. Four roster snapshots yielded 63 identities. Declared directory queues were processed for 47 reachable identities; 16 remained inaccessible because of authentication or network failures.

We distinguish three evidence levels. Level A contains raw input or output arrays that can be recomputed locally. Level B contains historical summaries whose reported values can be checked for consistency, but whose raw measurements are absent. Level C contains documentation, configuration, or current storage metadata that helps interpret an artifact. The analysis never promotes a Level B result to a newly reproduced experiment. Level A identifies recomputability, not necessarily authenticated input pairing, as cohort D illustrates.

B. INPUT PROVENANCE AND SEPARATION

Table 1 summarizes the scientific cohorts. Sequence A has 32 distinct image tensors, states, and noise tensors. Its state-source timestamps span 6.201 s. Sequence B has 32 distinct tensors of each type, with state-source timestamps spanning 9.710 s. Their input-file SHA-256 values begin with `7f3fa817` and `683de797`, respectively; full digests are supplied in the supplement. These adjacent observations are not 64 independent tasks.

The constructed 11-input fixture has only three distinct image tensors and nine distinct state vectors. Its first three state vectors are zero. The additional state-sweep entries have zero source timestamps. A separate 44-input file repeats the same 11 image-state-noise combinations four times. We retain these records as diagnostic evidence and exclude them from the recorded-observation count. The original full sensor bags and independent labels for the constructed states were not acquired.

TABLE 1. Evidence cohorts and their permitted interpretation.

Cohort	Contents	Analysis boundary
A	32 recorded inputs; 30×32 outputs	Primary paired-array analysis; one short sequence.
B	32 recorded inputs; 30×32 outputs	Separate sequence and archived confirmation runs.
C	11 constructed inputs; 30×32 outputs	Three distinct camera images plus state-sweep inputs; not 11 independent recordings.
J (July)	45 recording fixtures; 60×32 noise	Input identities verified; historical output/timing summaries only.
D (August)	115 identifiers; 60×32 outputs	Raw arrays with nominal ID pairing; no full input/noise digest.
E	32 inputs; 50×26 native outputs	Four-camera bottle checkpoint; full reference inputs, names and candidate noise; no candidate image digest.
F	32 inputs; 50×16 native outputs	Separate checkpoint; nominal order/seed linkage; no embedded input/noise digest.
Systems	Synthetic cache cases; one 160-cycle stream	Separate LingBot timing, allocator and rail observations; no whole-policy precision baseline.

The 60-step archive contains 32 calibration recordings with three inputs each, eight selection recordings with two inputs each, and 45 validation recordings with one input each. We verified all 85 fixture files against their dataset-manifest hashes. Recording names and exact image-state pairs do not overlap across these splits. All 45 acceptance fixture hashes also match the acquired inputs. The retained source manifest describes cup transfers relative to a tray, with calibration and selection covering both movement directions and validation covering one direction. Distinct files and byte identities do not establish broad task diversity or absence of prior developer exposure to validation outcomes.

A fourth-round bounded search of related existing workspaces added 16 sequence-B outputs and seven sequence-A verification outputs. Their embedded full fixture hashes match the original A/B inputs. Three of the seven A output pairs are aliases of earlier results; four are additional numerical variants. These records extend the retained search archive rather than its input population.

A separate August verification directory (cohort D) contains three raw output files with identical ordered lists of 115 sample identifiers and action shape 60×32. Their labels comprise 45 earlier validation recording/frame pairs, 60 additional inputs from 54 recording names dated July 21–22, and 10 inputs from five diagnostic scene names. There are 104 recording names in total. The first 45 identifiers match the earlier acceptance manifest's validation labels, so they add no independent validation recordings. Unlike A/B, D embeds

no complete input-file digest: matching identifiers support a nominal pairing, but do not authenticate identical images, preprocessing, or diffusion noise across runs. D is therefore reported separately with this weaker linkage, not pooled with the hash-matched evidence.

C. POLICY AND CONFIGURATIONS

The retained model manifest describes 27 vision layers, 18 language-encoder layers, 18 action-decoder layers, 32 action coordinates, and 10 denoising steps. The recorded observation tensors contain three red-green-blue (RGB) views, each with dimensions $3 \times 224 \times 224$. The 30-step and 60-step artifacts are treated as separate configurations. A longer action horizon is not interpreted as a repeated measurement of the shorter one.

For sequence A, the two primary references are the archived eager implementation and the static BF16 implementation. Component ablations are identified by their archived filenames and associated summaries. They include vision-only, encoder feed-forward network (FFN)-only, attention-only, and decoder-only changes, plus dynamic FFN and combined configurations. "Only" identifies the intended changed component in the archive; it does not certify an otherwise identical executable or clock state.

The retained 60-step profile definitions are more explicit. The dynamic FP8 profile uses FP8 paths for vision, encoder FFNs, encoder attention, and the decoder. The dynamic FP8/NVFP4 profile retains encoder FFNs in FP8 and uses NVFP4 for decoder FFNs. The static profile combines calibrated FP8 paths with AWQ NVFP4 encoder FFNs in layers 0–16. The final encoder layer's FFN is not executed in the retained graph. Calibration uses a scale margin of 1.1, with AWQ parameters $\alpha = 0.5$ and percentile 100. These are descriptions of retained implementations, not newly tuned settings.

D. NUMERICAL FIDELITY

Let $R, C \in \mathbb{R}^{N\times H\times D}$ denote paired reference and candidate output tensors. For A/B, comparisons require identical embedded full input hashes, dimensions, and ordering, linking the archived diffusion noise. D and F are separately identified nominal pairings. E retains full reference observations and exact candidate noise/name equality, with source-described replay ordering but no candidate digest of the complete observations. These linkage levels are never pooled. All metric arithmetic is recomputed in float64 from the saved arrays. The mean absolute error (MAE) is

$$\mathrm{MAE}(R, C) = \frac{1}{NHD} \sum_{i=1}^{N} \sum_{h=1}^{H} \sum_{d=1}^{D} |C_{ihd} - R_{ihd}|. \quad (1)$$

We also report root mean squared error, relative ℓ_2 discrepancy, coordinate error quantiles, and maximum absolute discrepancy. Aggregate cosine is

$$c_{\mathrm{all}} = \frac{\langle \mathrm{vec}(R), \mathrm{vec}(C) \rangle}{\| \mathrm{vec}(R) \|_2 \| \mathrm{vec}(C) \|_2}. \quad (2)$$

The minimum input cosine is $c_{\min} = \min_i c(R_i, C_i)$. Computing both values prevents a high-energy or well-matched part of the corpus from concealing a discrepant input. The script also preserves input-level and predicted-horizon metrics.

Normalized and denormalized action arrays are analyzed separately. The retained schema maps coordinates to trunk, left arm, right arm, head, left hand, and right hand groups. Native-coordinate plots describe the numerical values emitted by the policy transform. Complete physical unit and command-contract provenance is unavailable for every coordinate. Consequently, we do not pool these values into a physical tracking-accuracy claim or convert them into end-effector displacement.

For an interpretable numerical sensitivity check, we apply the archived aggregate floors $c_{\mathrm{all}} \geq 0.998$ and $\mathrm{MAE} \leq 0.03$, and an input floor $c_{\min} \geq 0.99$. These thresholds describe agreement with a reference implementation. They are neither task-success criteria nor physical safety limits. We also examine input floors of 0.98, 0.995, and 0.999.

E. NATIVE-ACTION REPLAYS AND NOMINAL KINEMATICS

Cohort E contains 32 saved observations, four $3 \times 480 \times 848$ uint8 images per input, 26 native action coordinates, and 50 predicted positions. A selected-episode list contains eight indices; per-input episode/frame membership is not serialized. The instruction concerns a left-hand bottle transfer between two coasters and return to rest. The stored reference labels its device as NVIDIA H20, PyTorch 2.7.1+cu126, and dtype as BF16. These historical labels are not independently authenticated hardware measurements. The contract lists seven revolute joints per arm and six hand coordinates per side. Retained replay source applies postprocessing, including restoration of relative arm actions to absolute targets. The original subset has five distinct NPZ candidate families with exactly equal reference noise and joint names. A later workspace adds 11 unguided candidate arrays with matching full reference-file digests in companion summaries; these reuse the same 32 observations and noise, while exact executable history remains incomplete. Configuration aliases are deduplicated in interpretation.

Cohort F is a separate three-camera checkpoint with 32 saved inputs, 50 predicted positions, seven joints and one gripper coordinate per arm. Its archived instruction concerns packing clothes and a box into a bag. The source converts gripper state widths to training fractions, describes the output arms in radians, and constructs per-input noise from a seed. Candidate files do not embed the full fixture/noise digest; filename/order/seed concordance therefore supports only a conditional comparison. Moreover, the current fixture's column 7 lies outside the width range assumed by the retained gripper conversion, leaving input-schema concordance unresolved.

For both cohorts, we independently parse the acquired Unified Robot Description Format (URDF) tree and compose joint-origin and axis-angle transforms in float64. Action-name mapping, rather than a shared slice assumption, distinguishes E's right-arm columns 7–13 from F's 8–14. For

arm flange frame e, let $T_e(q) = [R_e(q), p_e(q)]$ denote nominal forward kinematics. We compute

$$\begin{aligned} d_p &= 10^3 \|p_e(q_C) - p_e(q_R)\|_2 \quad (\mathrm{mm}), \\ d_R &= \angle\big(R_e(q_R)^\top R_e(q_C)\big)\,. \end{aligned} \tag{3}$$

with d_R reported in degrees. Unspecified common upstream joints are set to zero; a shared upstream rigid transform leaves both discrepancies invariant. The primary geometry is the acquired air-5 model for E and air-3 model for F. A second acquired model checks URDF-variant concordance for E. It yields identical discrepancies for the selected comparisons, without testing calibration or geometric uncertainty. Scalar Rodrigues calculations cross-check the batched implementation on 20 seeded poses per arm.

We report continuous discrepancies and the number of inputs whose two flange errors stay within an archived engineering tolerance of 3 mm and 0.5° at all 50 positions. Sensitivity to 1, 2, 3, 5, and 10 mm position tolerances is retained. The tolerance is not a validated task or safety criterion. Current nominal geometry is not proof of the historical calibration, and flange discrepancies of predicted targets are not physical tracking errors. No mesh, contact, compliance, or hand-object interaction is simulated.

F. LATENCY, REPETITION, AND RESOURCES

Each selected sequence-A output file contains 32 recorded latency values, one per input. We report the median, mean, standard deviation, empirical 95th and 99th percentiles (P95 and P99), and extrema. Percentiles use NumPy's default linear interpolation. The raw historical harness that produced every 30-step file is not fully preserved, so timer synchronization and preprocessing coverage cannot be established uniformly. These values are labeled recorded per-input latency, without asserting complete sensor-to-action latency or cold-start cost.

The retained 60-step acceptance implementation prepares the first input once, performs 10 warmup forward calls, and records 50 forward-call durations using CUDA events. Input preparation occurs before the timed interval. Fidelity is evaluated separately over all loaded inputs. Historical documentation reports MAXN operation on a Jetson Thor system; contemporaneous clock, temperature, occupancy, and power traces are absent. We therefore do not compare 30-step per-input timings numerically with 60-step CUDA-event timings as an architectural speedup.

Resource accounting uses the observed byte sizes of the current packed BF16 file and its quantized sidecar. Sizes are reported in decimal gigabytes. Neither a file size nor a nominal power setting measures peak graphics processing unit (GPU) allocation, resident memory, bandwidth, or energy. The current manifest postdates the July acceptance summaries, so the storage snapshot is reported separately from historical speed ratios.

For E, the archived harness times preparation, policy forward call, and postprocessing after reset, input clone, and initial synchronization. It cycles 32 observations but reuses the first input's noise for timing; fidelity replay uses each input's stored noise. Its 200-iteration comparisons and 150-iteration graph ladder have 10 warmup calls. Only rounded summaries survive. We call this a prepared-call boundary, although the files label it "end-to-end." E's PyTorch peak allocation counters are incomplete for TensorRT allocations. F supplies nine 20-value timing vectors rounded to 0.1 ms; these repeat a single prepared synthetic observation and are kept distinct from F's 32-real-input timing summaries.

G. SEPARATE CACHE AND COMPLETE-CYCLE RESOURCE CASE

The LingBot archive contains three formal BF16-query attention cases with batch/query pairs $(1, 24)$, $(2, 24)$, and $(2, 240)$, each with 9,504 historical tokens, 24 heads, head dimension 128, and page size 16. The retained script generates seeded random tensors. The archived BF16/FP8-E4M3 cache comparisons use FlashInfer 0.6.17, the fa2 backend, and five timing samples, each averaging 200 CUDA-event iterations after 50 warmup calls. We recompute timing summaries from those five samples and inspect reported attention-output discrepancies; no raw attention tensors are available for re-evaluation. Different-mask, query-quantized, smoke, and profiler runs are kept outside this comparison.

For one separately archived FP8-core/BF16-cache streaming execution, we verify the observation and asset-manifest digests, all 160 saved action-array hashes, and cycle timestamps. Duplicate rank-level reports are counted once. The archive labels cycles 60–159 as steady; previous model outputs supply state backfill for timing only. No live robot I/O or command publication occurred in this benchmark. We report cycle, inference, and key–value (KV) update durations, plus PyTorch allocator counters. These counters are not a complete census of TensorRT or device-wide memory allocation.

The same stream retains 4,151 timestamped `tegrastats` observations. NVIDIA distinguishes instantaneous rail power from its running average [17]. We integrate the instantaneous `VDD_GPU` field over each fully covered steady-cycle timestamp interval, interpolating at its boundaries and applying the trapezoidal rule. A left-held integration checks interpolation sensitivity. These estimates include idle and other users' GPU-rail load, without idle subtraction, calibrated sensor response, or whole-board coverage. They measure one archived operating configuration and cannot establish quantization energy savings. Complete methods and accounting quantities are retained in the supplement.

H. STATISTICAL SCOPE AND REPRODUCIBILITY

The unit for primary discrepancy analysis is a paired recorded input, with temporal dependence within each sequence. Horizon positions and action coordinates remain nested within an input. We report finite-corpus descriptive effects, without treating coordinates as independent observations or assigning population-level significance to configuration differences.

Robustness is assessed by deleting aligned contiguous blocks of one, four, or eight inputs from sequence A and recomputing MAE. The resulting range measures sensitivity

to the retained sequence composition; it is not a confidence interval. For repeated historical acceptance files, we report the repeat range rather than multiplying the number of recording inputs by the number of repeats. The analysis script records its own digest and the Python, NumPy, SciPy, and Matplotlib versions. It loads arrays with object deserialization disabled and never imports the robot runtime.

IV. RESULTS

A. REFERENCE CHOICE REVEALS IMPLEMENTATION DISCREPANCY

The eager and static BF16 outputs differed on sequence A despite the shared input hash. Their aggregate normalized cosine was 0.999198, and normalized MAE was 0.009347. Minimum input cosine was 0.987991. The maximum native-coordinate discrepancy was 0.049805. This establishes a numerical implementation discrepancy before low-precision configurations are evaluated; it does not identify a faulty implementation.

The recorded medians were 249.90 ms for eager inference and 109.87 ms for static BF16. Their ratio was 2.275. Because the implementations differ, this ratio should not be attributed to quantization. Subsequent primary fidelity results use static BF16 as the reference, while the supplied tables retain eager-reference metrics for comparison.

B. AGGREGATE AGREEMENT MISSES AN INPUT-LEVEL REGRESSION

Table 2 and Fig. 1 show that the largest recorded latency reductions also accompanied substantial fidelity changes. The dynamic and static mixed configurations reached 81.76 and 67.70 ms, respectively. Relative to BF16, their minimum input cosines fell to 0.928032 and 0.965292. Five dynamic-mix inputs and two static-mix inputs were below 0.99. Neither combined configuration met the aggregate cosine floor.

The vision-only result illustrates a different limitation. Its aggregate cosine of 0.999560 and MAE of 0.005076 both met the aggregate thresholds. One input nevertheless had cosine 0.987160, below the input floor. The discrepancy was visible only after preserving input-level metrics. The associated 105.63 ms median was only 3.86% below the BF16 median, indicating a small recorded timing benefit alongside a localized numerical regression.

Dynamic FFN and decoder NVFP4 retained minimum input cosines above 0.99 on this sequence. Their recorded medians were 102.39 and 101.30 ms, while normalized MAEs were 0.008535 and 0.015921. These are sequence-specific numerical trade-offs; task success was not measured.

An exploratory screen of all 281 primary-directory records with sequence A's hash identified 257 distinct normalized/native tensor pairs. Eight pairs passed aggregate thresholds but failed the minimum-input floor. The supplement retains all counts and aliases; adaptive archival selection prevents an unseen-test interpretation.

C. DISCREPANCY IS HETEROGENEOUS ACROSS INPUTS AND COORDINATES

Figure 2(a) localizes the strongest mixed-precision disagreement near input 12. Across native-coordinate groups, the arms and right hand had larger discrepancies than the trunk, head, and nearly constant left-hand outputs. This heterogeneity makes a single pooled native-coordinate error insufficient for interpreting the control implications.

The horizon profiles in Fig. 2(c) showed different discrepancy levels across configurations. Errors within a predicted chunk are not evidence of temporal error accumulation during physical execution. The observed pattern can reflect the policy's computation and its action normalization, without any closed-loop rollout having occurred.

Deleting one input left dynamic-mix MAE between 0.020858 and 0.023383, compared with its full-sequence value of 0.022945. Static-mix MAE remained between 0.021668 and 0.022873. Thus, the aggregate discrepancy persisted when the most influential individual input was removed. Eight-input block deletion produced wider ranges of 0.017984–0.026102 and 0.019930–0.024622, respectively. The wider block ranges expose dependence on sequence composition.

D. REPEATED OUTPUTS AND RECORDED TIMING VARY DIFFERENTLY

Sequence B was evaluated separately because its input hash and timestamps differ. Three named dynamic-FFN confirmation files contained exactly equal normalized and native output arrays. Their recorded median latencies ranged from 100.43 to 100.66 ms. Other archived executions with those same output arrays had medians up to 114.99 ms. This pattern separates numerical repeatability from timing repeatability under incompletely documented conditions.

The archive also contained two sequence-A decoder files with exactly equal output arrays but different timing vectors. Their medians were 101.30 and 115.57 ms. They are retained as a timing-provenance warning, rather than used as evidence that equivalent numerical results imply equal runtime cost. Without a complete contemporaneous configuration record, environmental effects and implementation differences cannot be separated.

E. ADDITIONAL ARCHIVES SHOW FAVORABLE PRECISION CONFIGURATIONS

Sequence B's newly recovered workspace contains a BF16 ablation and three-repeat dynamic-FP8, static-FP8, and AWQ-labeled families (Table 3). Each repeat family has exactly equal saved outputs across its three files. All three families met the aggregate and minimum-input numerical thresholds against this workspace's BF16 reference. Their lowest minimum input cosine was 0.995538 for AWQ. Thus, the adverse sequence-A mixed configurations do not support a general claim that mixed precision fails the numerical gate. The B workspace has different outputs and an incompletely

TABLE 2. **Recomputed sequence-A results for the same 32 inputs and fixed noise. Numerical discrepancies use static BF16 as reference. Each latency vector contains 32 recorded per-input values; these are descriptive order statistics.**

Archived configuration	Median (ms)	P95 (ms)	Speed ratio	Cosine	MAE	Min. input cosine	Inputs below 0.99
Eager	249.90	254.38	0.440	0.999198	0.009347	0.987991	1
BF16	109.87	110.32	1.000	1.000000	0.000000	1.000000	0
Vision only	105.63	108.98	1.040	0.999560	0.005076	0.987160	1
Encoder FFN only	101.63	102.46	1.081	0.996780	0.019490	0.978087	2
Attention only	110.51	113.14	0.994	0.999252	0.009210	0.992153	0
Dynamic FFN	102.39	103.08	1.073	0.999420	0.008535	0.996273	0
Decoder NVFP4	101.30	101.98	1.085	0.998229	0.015921	0.993852	0
Dynamic mix	81.76	81.94	1.344	0.995056	0.022945	0.928032	5
Static mix	67.70	68.00	1.623	0.996464	0.022513	0.965292	2

Speed ratio = BF16 median / configuration median. Eager and BF16 differ in implementation while retaining BF16 arithmetic. "Only labels identify archived component ablations, not a new algorithm. Full filenames, P99, extrema, and both reference choices are retained in the supplement.

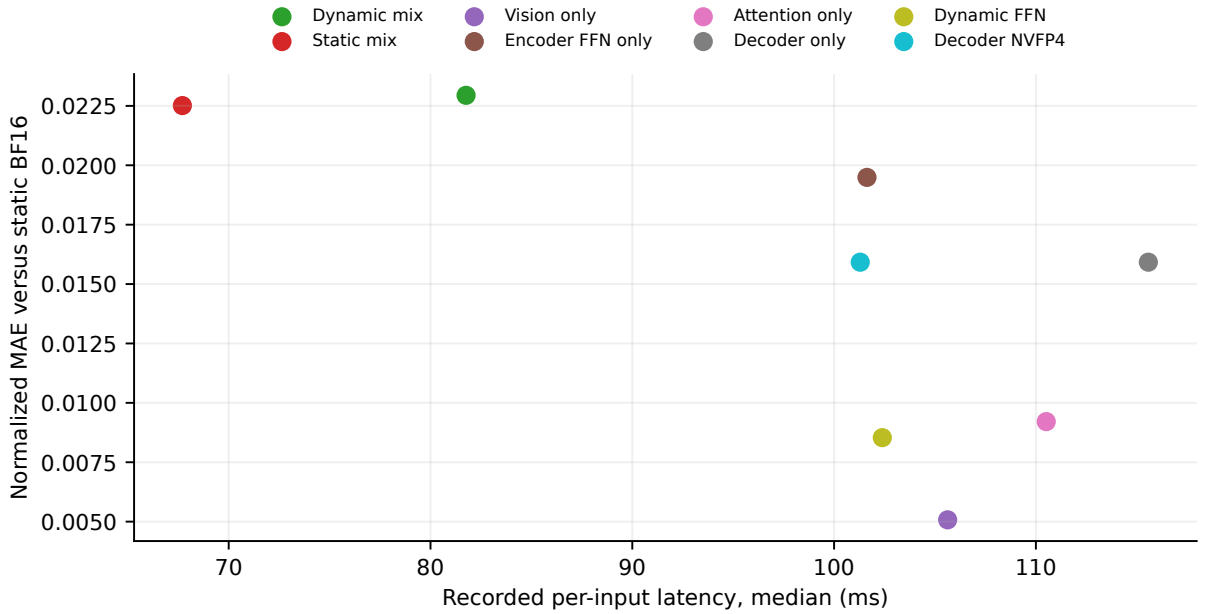


FIGURE 1. **Sequence-A latency–fidelity comparison. Each point uses the same 32-input fixture and its archived noise. The horizontal axis is the median of a saved per-input timing vector, and the vertical axis is recomputed normalized MAE relative to static BF16. Points are archived configurations, not independent repeated experiments. Unknown historical clock and harness differences limit causal interpretation of latency rankings.**

TABLE 3. **Additional whole-policy replay evidence. B uses its hash-matched BF16 ablation; D uses fresh PyTorch. H60 D has ID-matched arrays, with input linkage audited separately. Values are recomputed, not copied from the nonconcordant report.**

Configuration	Median ms	MAE	Cosine	Min. input
B: BF16	128.37	0	1	1
B: Dynamic FP8	82.30–82.64	0.00883	0.999559	0.997630
B: Static FP8	64.97–65.48	0.00807	0.999704	0.998757
B: Static AWQ	56.59–57.53	0.01377	0.999007	0.995538
D: Fresh PyTorch	294.30	0.00000	1.000000	1.000000
D: Static BF16	132.19	0.00430	0.999927	0.999560
D: Dynamic FP8	92.81	0.00647	0.999881	0.999461

B: 32 inputs, H30; ranges cover three saved execution medians with identical outputs. D: 115 ID-matched inputs, H60; 104 recording names, with dependence within task/scene. No cross-cohort speed ratio is implied.

frozen revision; its contrast with A cannot isolate observation difficulty, calibration, or implementation as a cause.

The 115-input H60 archive also retained favorable outputs (Fig. 3). Dynamic FP8 had normalized MAE 0.006468 and minimum input cosine 0.999461 against fresh PyTorch; static BF16 had MAE 0.004301 and minimum cosine 0.999560. Relative to static BF16, dynamic-FP8 MAE was 0.004194 and minimum cosine was 0.999516. We retain both references and all three recording strata in the supplementary tables. These are recomputed differences between nominally ID-paired arrays, subject to D's missing input-digest linkage.

Reanalysis also exposed a version inconsistency. The retained report lists a PyTorch reference median of 154.77 ms, whereas the current `pytorch_bf16_fresh.npz` timing vector has median 294.30 ms. Its candidate-versus-reference discrepancy values also differ from the recomputed values. The static-BF16 and dynamic-FP8 medians of 132.19 and 92.81 ms agree with their saved arrays. We use the current

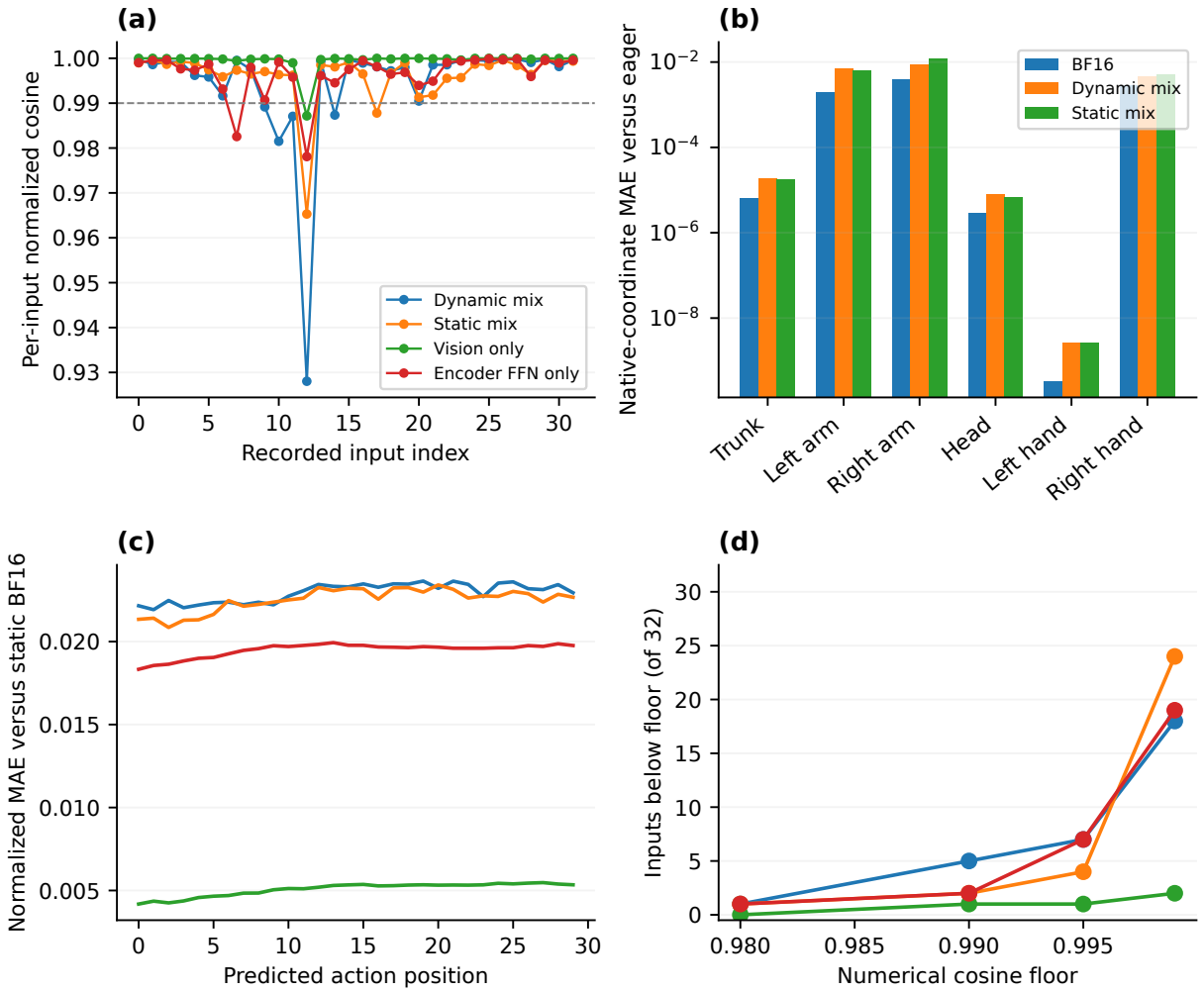


FIGURE 2. **Error structure on sequence A. (a) Per-input normalized cosine against static BF16; the dashed line is the archived numerical floor. (b) Native-coordinate group MAE against eager outputs on a logarithmic scale; group values do not imply commensurate physical units. (c) Mean normalized discrepancy at each predicted action position against static BF16. These positions are outputs of one chunk, not realized future robot states. (d) Counts below alternative input cosine floors. No panel contains independent task-success measurements or inferential error bars.**

raw arrays throughout and do not attach the earlier report's speed ratio or discrepancy scores to the fresh reference. A file named "fresh" is not sufficient to establish historical run concordance.

F. THE 60-STEP ARCHIVE HAS A DISTINCT ACCEPTANCE BOUNDARY

The acquired 60-step fixtures verified the stated 96/16/45 calibration, selection, and validation input counts. The acceptance summary's 45 fixture hashes matched the validation copy. Its four-profile results are shown in Table 4 as historical reported values. No raw outputs or timing vectors for this July acceptance run were recovered, so its fidelity values remain historical summaries.

The archived static FP8/AWQ NVFP4 profile had an aggregate cosine of 0.999276, MAE of 0.012234, and minimum input cosine of 0.990386. Its median CUDA-event latency was 53.31 ms, relative to 119.95 ms for static BF16. Three additional repeats preserved the same reported fidelity and yielded median speed ratios of 2.2379–2.2513. These repeats reused the same validation fixtures and therefore do not constitute 135 independent task trials.

Source inspection showed that timing repeatedly evaluated only the first prepared input. The 45-recording count belongs to fidelity evaluation, while the 50 timing iterations belong to one input. Figure 4 preserves this distinction. The static profile's minimum input cosine exceeded the archived floor by only 0.000386. This narrow numerical margin merits attention even though the historical acceptance flag was true.

G. NATIVE-OUTPUT COSINE CAN CONCEAL MILLIMETRE-SCALE FLANGE DIFFERENCES

Cohort E's local PyTorch candidate had aggregate native cosine 0.999999760 against the stored H20-labelled reference, yet its maximum left-flange difference was 3.068 mm and 0.510° (Table 5). The TensorRT split candidate had cosine

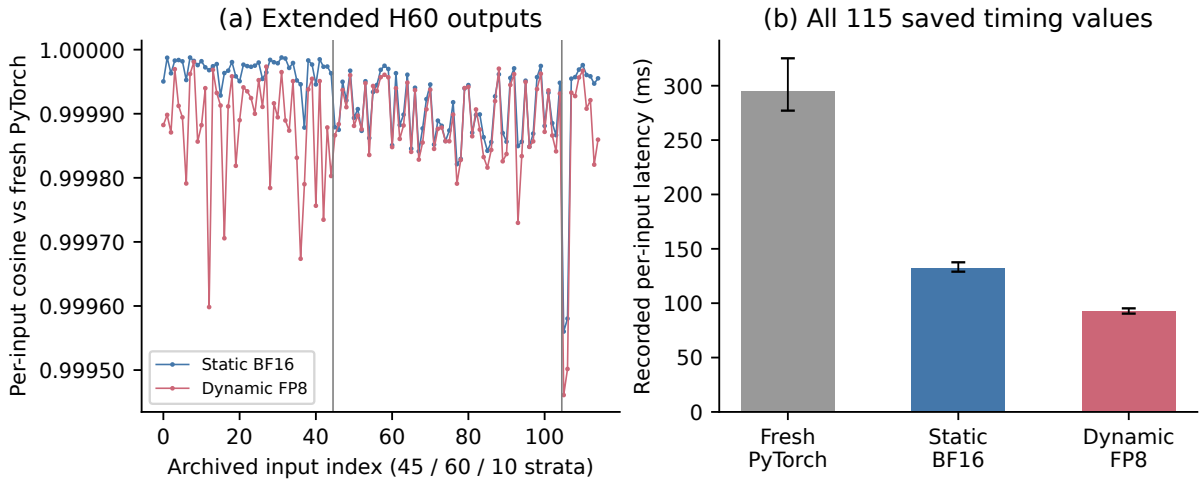


FIGURE 3. **The additional H60 verification archive. (a) Recomputed input cosines against the acquired fresh-PyTorch arrays, ordered as 45 earlier validation identifiers, 60 additional recording-labeled inputs, and 10 scene diagnostics. Vertical separators mark these strata. Pairing uses identifiers, with no embedded full input digest. (b) Medians of 115 recorded per-input durations; whiskers show observed minima and maxima, not confidence intervals. The older report's PyTorch reference differs and is excluded.**

TABLE 4. **Historical 60-step acceptance summary, transcribed from the archived JSON and not remeasured. Fidelity covers 45 recording fixtures; timing covers 50 repetitions of the first prepared input after 10 warmup calls.**

Profile	Median (ms)	P95 (ms)	P99 (ms)	Cosine	MAE	Max. error	Min. input cosine
Static BF16	119.95	132.76	133.15	1.000000	0.000000	0.000000	1.000000
Dynamic FP8	80.71	81.71	81.92	0.999905	0.004852	0.150391	0.998878
Dynamic FP8 + decoder NVFP4	77.12	78.05	78.32	0.999805	0.009396	0.167969	0.999025
Static FP8 + encoder AWQ NVFP4	53.31	54.10	54.35	0.999276	0.012234	0.347656	0.990386

Errors are in normalized action coordinates relative to static BF16. Raw output tensors and timing vectors for this July acceptance were not recovered; cohort D is a separate later archive. The retained implementation defines the timer scope; exact run-to-source revision identity is not established.

TABLE 5. **Native-action and nominal kinematic discrepancies. Each cohort has 32 inputs and 50 predicted positions. E uses the stored H20-labelled reference with 26 coordinates; F uses the JAX-labelled reference with 16 coordinates. Geometry is a separately frozen current URDF. F also has an unresolved fixture/preprocessing column conflict.**

Cohort	Archived configuration	Pooled cosine	Min. input cosine	Left max. (mm)	Right max. (mm)	Left max. (°)	Right max. (°)	Inputs within gate
E	PyTorch BF16 label	0.999999760	0.999999302	3.068	0.004	0.510	< 0.001	29/32
E	PyTorch FP32 label	0.999999605	0.999998909	4.750	0.006	0.571	0.001	27/32
E	TensorRT split	0.999999645	0.999999168	4.867	0.006	0.596	0.001	21/32
E	Graph + vision BF16	0.999999639	0.999999156	3.541	0.005	0.549	0.001	27/32
F	PyTorch BF16 label	0.999889956	0.998246877	6.487	9.041	0.613	1.440	26/32
F	TensorRT FP16 label	0.999953065	0.999124130	7.170	8.575	0.523	1.486	23/32
F	TensorRT FP8 label	0.999837586	0.998233708	19.926	15.314	2.074	2.538	3/32

Gate: both flanges remain within 3 mm and 0.5° at every predicted position for that input. This archived engineering tolerance has no demonstrated task-success or physical-safety meaning. Values describe nominal forward kinematics of saved targets, not measured motion. Precision labels do not certify every operator's dtype or isolate quantization from backend changes.

0.999999645, with 4.867 mm and 0.596° maxima. The corresponding right-arm position maxima were 0.004 and 0.006 mm: the instruction activates the left arm, while the nearly unchanged right arm and other coordinates contribute to pooled agreement. All E candidates in the table exceeded minimum input cosine 0.9999989. Even this input-level similarity alone did not imply submillimetre nominal flange agreement.

Across the 32 inputs, 29 local-PyTorch and 21 split candidates stayed within the joint flange tolerance at every predicted position. These counts describe numerical agreement, not successful episodes. Relative to the local PyTorch outputs instead of the cross-host reference, the split candidate reached 5.314 mm and 0.650°, with 24 inputs within the same tolerance. The FP32-labelled candidate did not uniformly reduce discrepancy to a BF16-labelled reference: its left-flange maximum was 4.750 mm. Reference agreement is therefore not monotone in a precision label.

F's FP8-labelled TensorRT outputs had pooled cosine 0.999838 and minimum input cosine 0.998234, while nominal left/right position maxima reached 19.926/15.314 mm and rotation maxima reached $2.074^\circ/2.538^\circ$. Only three inputs remained within the archived flange tolerance. All values use

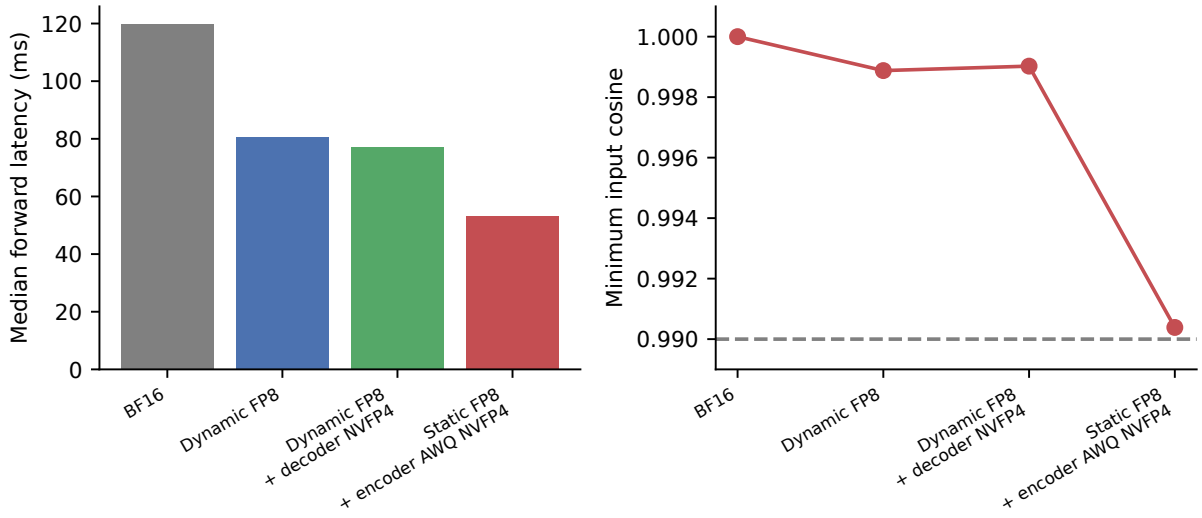


FIGURE 4. Historical 60-step summaries. Left: archived median CUDA-event durations from 50 repetitions of one prepared input. Right: archived minimum input cosine over 45 recording fixtures. Both panels transcribe existing summaries, rather than newly measured inference. The dashed line is the historical 0.99 numerical floor. Model setup, sensor acquisition, preparation, and robot execution are outside the documented timer.

the acquired raw outputs; the retained BF16 flange summary is inconsistent with that current comparison and is excluded. The BF16- and FP16-labelled alternatives retained 26 and 23 inputs, respectively. This comparison is conditional: backend/export effects and missing input linkage remain unresolved. The current fixture also has column-7 values from -2.232 to -1.387, whereas retained preprocessing treats that column as a gripper width in $[0, 0.09]$ m. Without an authenticated run revision, these values cannot be interpreted as representative valid-input quantization behavior. The discrepancy motivates an input-schema check before attributing an adverse result to precision.

H. A FAVORABLE SPEED COMPARISON CHANGES WITH ITS BASELINE

E's archived split-backend prepared-call median was 209.31 ms, 27.16% below the PyTorch value of 287.35 ms. Reported PyTorch peak allocation increased from 8.16 to 8.84 GiB; this is a counter comparison, not total TensorRT memory use. Split-backend acceleration therefore does not establish a memory saving.

The graph ladder exposes a second baseline issue. With full graph, batched vision, and scaled dot-product attention (SDPA), the archived median was 513.01 ms. Adding vision BF16 gave 290.36 ms, a 43.40% reduction within that ladder, but 1.05% above the unaccelerated PyTorch median. Retained acceleration source describes the default vision tower/projector as FP32 and explicitly casts them for the vision-dtype option; the checkpoint's BF16 field does not describe every tensor. Full graph alone and graph plus vision batching had exactly equal saved actions and medians of 534.57 and 534.31 ms. These are historical configuration observations with uncertain run revisions and clock conditions, not newly isolated causal effects. The complete graph ladder and URDF-variant concordance are retained in the supplement.

The later E workspace links 11 unguided output files to the same complete reference package through companion digests and explicit noise. They contain four distinct output tensors, without adding inputs. With the FP16-cache flag, the recorded median changed from 175.27 to 173.54 ms, while the PyTorch process-peak counter increased from 8.845 to 9.204 GiB. Both configurations retained 24 of 32 inputs within the flange tolerance. These later timers include transfer of results to the CPU, and their counters can include loading; they are not pooled with the earlier E measurements. The complete screen and exclusions are supplied in the supplement.

I. RETAINED QUANTIZED WEIGHTS ADD STORAGE IN THE CURRENT SNAPSHOT

The current packed BF16 file occupied 6,706,757,645 bytes, while the encoder AWQ NVFP4 sidecar occupied 963,271,463 bytes. Retaining both required 7,670,029,108 bytes, an increase of 14.36% relative to the packed file alone (Fig. 6). The raw source checkpoint occupied 8,287,136,146 bytes, but changing serialization or packing is a different comparison from adding a sidecar to an already packed baseline.

The manifest identifies an SM110/CUDA13/Python 3.12 native-operations interface and records the sidecar hash. It does not provide measured peak GPU memory or energy. Moreover, its modification time follows the historical acceptance records, and its sidecar calibration sample count differs from the earlier calibration description. We therefore report the byte inventory as a current storage result, without pairing it with a historical speed ratio to construct a jointly verified Pareto point.

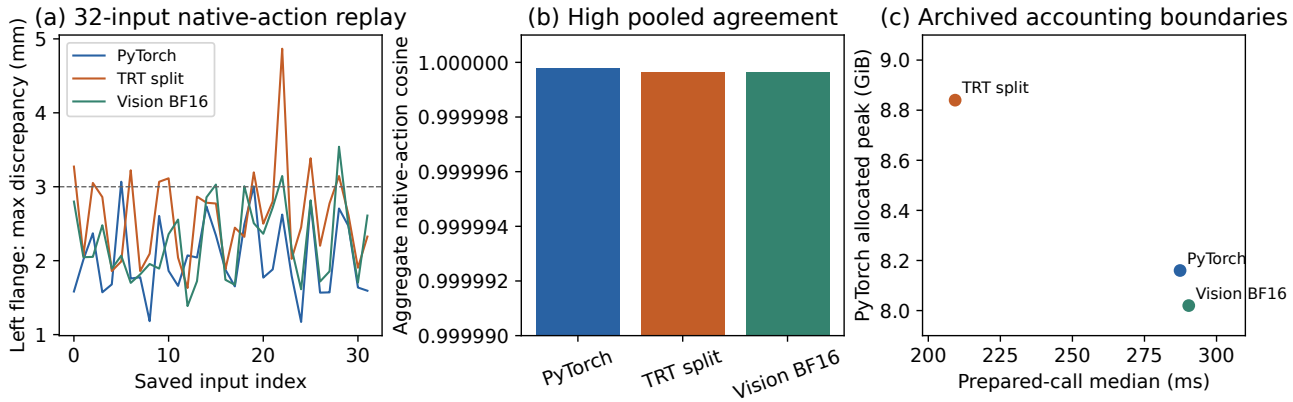


FIGURE 5. **Cohort E. (a) Each input's largest predicted left-flange position discrepancy relative to the stored H20-labelled reference; the dashed line is an engineering tolerance. (b) Corresponding aggregate native-action cosine. (c) Archived prepared-call medians and PyTorch allocation peaks for three configurations. Timing has no raw vector here, and allocation counters do not cover every backend. No panel reports physical tracking, task success, or independent deployments.**

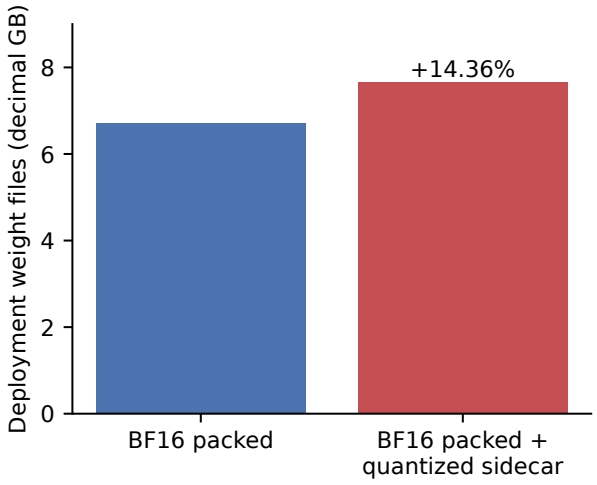


FIGURE 6. **Current serialized weight-file storage. The BF16 packed file remains present alongside the quantized encoder sidecar. Small shared metadata, normalization statistics, and tokenizer files are excluded from both bars. This is a storage snapshot, with no inferred GPU-memory or energy saving. Its exact identity with the historical acceptance artifact is unverified.**

J. CACHE COMPRESSION AND COMPLETE-CYCLE COSTS HAVE DIFFERENT SCOPES

FP8 KV storage halved the logical tensor bytes in each synthetic attention case, before page padding and workspace costs. Yet median attention latency decreased by 12.5% and 10.9% for the two 24-query cases and increased by 14.0% for the 240-query case (Fig. 7). Including quantization in the append timer also cost more than appending prequantized values. Reported synthetic attention cosines were 0.999243–0.999264; these are not whole-policy action similarities. The cases demonstrate a shape-dependent latency tradeoff despite identical nominal byte reduction.

The separate streaming configuration used an FP8 core with BF16 paged caches. Its 100 steady cycles had median complete-turn duration 2.838 s (P95 3.013 s), compared with median inference duration 2.023 s. Mean KV-update time accounted for 27.99% of mean turn duration. Every steady turn exceeded the archive's declared 0.8-s action horizon, demonstrating a mismatch in this offline schedule, without evaluating asynchronous control or physical deadline failures.

The archive reported peak PyTorch allocated and reserved memory of 21.40 and 22.81 GiB, respectively. Integration of the existing GPU-rail observations yielded median 96.36 J per steady turn, with range 91.92–101.92 J and P95 99.58 J. Time-weighted GPU-rail power over those turns was 33.91 W. Alternative left-held integration changed individual estimates by at most 0.73%. Sampling and sensor limitations remain larger unresolved uncertainties; this sensitivity is not an accuracy guarantee. No matched full-model BF16 run is present, so these observations establish neither memory reduction nor energy savings from quantization.

V. DISCUSSION

A. WHAT SHOULD GUIDE A PRECISION DECISION?

Precision decisions depend on the reference implementation, input-level discrepancy, timing scope, and resource category. Favorable B/D configurations qualify the adverse A examples. The native-action E/F comparisons further show that pooled or input-level cosine can coexist with meaningful nominal flange differences. Reference mismatches, baseline choice, and shape-dependent cache costs limit conclusions drawn from one similarity value or speed ratio.

The input-level result supports examining the decision-sensitive states motivating SQIL and QVLA [3], [4]. Our numerical outlier does not identify a task-critical state or reproduce their closed-loop failure mechanisms; it identifies an input requiring further evaluation.

Reference choice also changes the interpretation of error. A BF16 implementation is a computational reference, not a ground-truth policy. E's FP32-labelled candidate does not consistently approach the BF16-labelled reference more closely, reinforcing that agreement and physical accuracy are different objectives. Disagreement between eager and

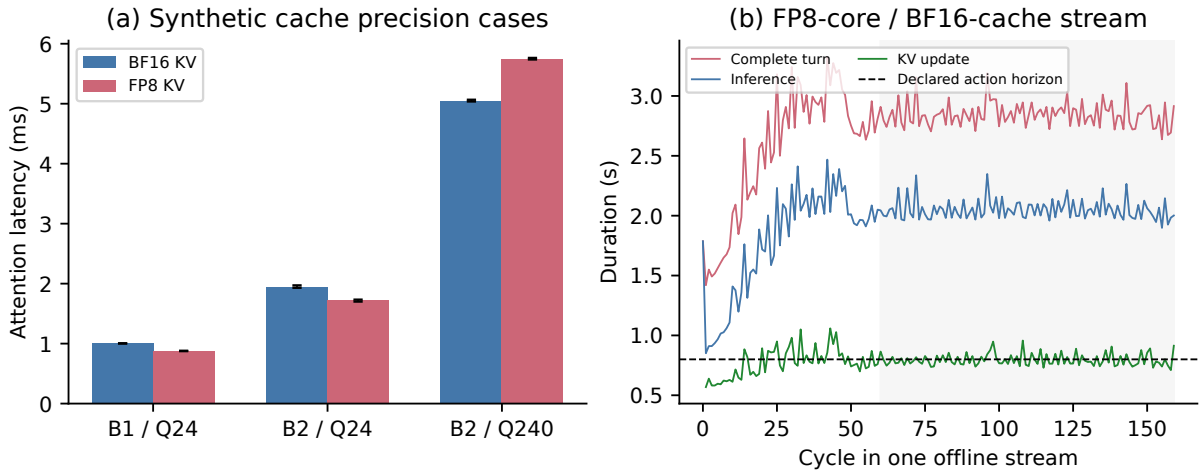


FIGURE 7. **Independent LingBot resource evidence with separate workloads. (a) Synthetic attention timing medians; whiskers span five saved repeat means. FP8 denotes KV precision with BF16 queries; each cache's logical bytes halve, but latency direction depends on shape. (b) One FP8-core/BF16-cache offline stream, with the archive's steady region shaded. KV update is additional to inference. The dashed line is the configured action horizon. The kernel cases were not substituted into this stream, so no whole-policy speedup is inferred from panel (a).**

static BF16 may reflect kernel arithmetic, graph organization, preprocessing, or other implementation details. The retained records do not uniquely resolve those causes. Reporting both references makes the ambiguity visible and prevents attributing all eager-relative discrepancy to quantization. The H60 report-to-file mismatch further shows why even a retained summary needs to be linked to an immutable reference artifact.

B. ARITHMETIC PRECISION AND SYSTEM RESOURCE COST

Resource categories can move in different directions. Retaining a sidecar increases serialized storage, while E's faster split configuration increases the reported PyTorch allocation peak. LingBot's cache byte reduction also coexists with a slower attention shape. These observations make the measurement boundary central to interpretation: file storage, logical cache bytes, allocator counters, total device memory and rail energy are distinct quantities.

Published precision scopes and timers differ across HoloQ-VLA, CHASE-VLA, and this archive [5], [6], preventing a numerical ranking. Our retained-file boundary instead informs distribution size, update transfer, and rollback capacity.

C. REPETITION SHOULD PRESERVE ITS EXPERIMENTAL UNIT

The distinction between fidelity coverage and repeated timing preserves the experimental unit emphasized by benchmarking guidance [14], [15]. Recording inputs, timing iterations and acceptance repeats characterize different variation; none supplies a count of successful robot episodes.

For practical reporting, an inference result should identify the reference executable, exact replay inputs, action shape, random noise, timer scope, timing-input count, and resource category. An aggregate fidelity value should be accompanied by a minimum or distribution over inputs. A low-latency configuration that crosses a numerical floor should remain visible, including when it is excluded from a recommended profile. These requirements arise directly from ambiguities observed in this archive.

D. LIMITS AND THE NEXT DISCRIMINATING EVALUATION

The hash-matched primary arrays cover two short recorded sequences in a narrow cup-manipulation context. Their temporal dependence limits population inference, and constructed fixtures add no independent task evidence. Cohort D broadens recording labels but lacks complete input/noise digests and overlaps the historical 45-recording validation labels. Neither archive establishes an untouched test set for this retrospective study. The LingBot stream is one state-dependent offline execution, not a set of independent robot trials.

The July acceptance archive lacks raw 60-step output/timing vectors and contemporaneous power traces; D supplies later raw arrays rather than recovering that run. Complete source-to-run revision links remain missing. LingBot supplies existing allocator and GPU-rail telemetry, with no matched baseline or independently calibrated whole-device measurement. Source checkpoint and sidecar hashes are recorded in the current manifest, but the large weight files were not reacquired and rehashed. Original hardware attribution for each 30-step file also remains unverified beyond the associated deployment records. A later manifest revision prevents proving that current storage and earlier acceptance measurements describe exactly one immutable artifact. These gaps limit reproducibility of inference, while leaving the local reanalysis of the acquired arrays reproducible.

No closed-loop rollout, task-success label, physical tracking measurement, or safety outcome is available for the selected comparisons. Output similarity can diagnose numerical

change, but does not establish equivalent behavior under contacts, delays, or observation drift. The numerical floors have no validated mapping to physical task tolerance. Nominal flange geometry improves interpretability in robot units while leaving calibration, tracking and contact unresolved. E lacks per-input episode membership; F additionally lacks full input/noise linkage and has a fixture–preprocessing conflict. Finding these files on other computers does not supply independent closed-loop trials.

The next evaluation should freeze an immutable artifact and common timing harness, retain input/noise digests and synchronized resource traces, and use previously unexposed recordings from multiple tasks. A separately authorized closed-loop study could then test whether numerical outliers predict task degradation. These observations would connect replay agreement with practical policy equivalence.

VI. CONCLUSION

A retrospective analysis of archived robot policy inference exposed distinctions that materially affect precision decisions. Hash-linked inputs separated recorded observations from constructed or repeated fixtures. Recomputed results showed that aggregate cosine could conceal a discrepant input, while a BF16 implementation change already produced measurable output differences. Historical acceptance summaries and retained source further separated multi-input fidelity evaluation from repeated single-input timing. Native-action replays added nominal flange discrepancies that remained visible even when input cosines were close to one; a component-precision speed ratio also changed interpretation when compared with the ordinary PyTorch baseline.

Retaining a quantized sidecar increased serialized storage, a cost distinct from runtime memory or energy. The study provides a reproducible finite-corpus evaluation of mixed-precision deployment evidence. Favorable B/D configurations qualify the adverse A results, and a report-to-reference mismatch shows why summaries need immutable linkage. The separate cache and stream evidence distinguishes logical compression from kernel latency, allocator peaks, and GPU-rail energy. Its conclusions remain retrospective; closed-loop task performance and comparative energy savings are unresolved.

DATA AND CODE AVAILABILITY

The local research package contains analysis scripts, generated metric tables, figure sources, acquired replay arrays, and SHA-256 provenance manifests. Original robot observations, checkpoint artifacts, and implementation files are organization-controlled materials. Public redistribution permission has not been established, and no public repository or accession is claimed. The analysis can be rerun against the retained local evidence copy. Access to underlying organization-controlled materials requires authorization from the data owner.

REFERENCES

[1] M. J. Kim, K. Pertsch, S. Karamcheti *et al.*, "OpenVLA: An Open-Source Vision-Language-Action Model," *arXiv preprint arXiv:2406.09246*, 2024. [Online]. Available: https://arxiv.org/abs/2406.09246

[2] Physical Intelligence, K. Black *et al.*, "$\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization," *arXiv preprint arXiv:2504.16054*, 2025. [Online]. Available: https://arxiv.org/abs/2504.16054

[3] S. Park, H. Kim, S. Kim, W. Jeon, J. Yang, B. Jeon, Y. Oh, and J. Choi, "Saliency-Aware Quantized Imitation Learning for Efficient Robotic Control," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 13 140–13 150. [Online]. Available: https://openaccess.thecvf.com/content/ICCV2025/html/Park_Saliency-Aware_Quantized_Imitation_Learning_for_Efficient_Robotic_Control_ICCV_2025_paper.html

[4] Y. Xu, Y. Yang, Z. Fan, Y. Liu, Y. Li, B. Li, and Z. Zhang, "QVLA: Not All Channels Are Equal in Vision-Language-Action Model's Quantization," *arXiv preprint arXiv:2602.03782*, 2026. [Online]. Available: https://arxiv.org/abs/2602.03782

[5] X. Wang, M. Li, S. Lyu *et al.*, "HoloQ-VLA: Uniform W4A4 Quantization of Vision-Language-Action Models," *arXiv preprint arXiv:2605.28803v3*, 2026. [Online]. Available: https://arxiv.org/abs/2605.28803v3

[6] J. Hyun, J. G. Min, G. Jung, and Y. Lee, "CHASE-VLA: Post-Training Quantization Framework for Vision-Language-Action Models with Chunk-Aware Scale Estimation," *arXiv preprint arXiv:2610.02666*, 2026. [Online]. Available: https://arxiv.org/abs/2610.02666

[7] L. Li, Q. Zhang, Y. Luo *et al.*, "Causal World Modeling for Robot Control," *arXiv preprint arXiv:2601.21998v2*, 2026. [Online]. Available: https://arxiv.org/abs/2601.21998v2

[8] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," *arXiv preprint arXiv:2210.17323*, 2022. [Online]. Available: https://arxiv.org/abs/2210.17323

[9] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration," in *Proceedings of Machine Learning and Systems*, 2024. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2024/hash/42a452cbafa9dd64e9ba4aa95cc1ef21-Abstract-Conference.html

[10] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," in *Proceedings of the 40th International Conference on Machine Learning*, 2023, pp. 38 087–38 099. [Online]. Available: https://proceedings.mlr.press/v202/xiao23c.html

[11] Triton Project, "Block scaled matrix multiplication," 2026, accessed: Oct. 19, 2026. [Online]. Available: https://triton-lang.org/main/getting-started/tutorials/10-block-scaled-matmul.html

[12] NVIDIA, "Working with quantized types," 2026, tensorRT documentation. Accessed: Oct. 19, 2026. [Online]. Available: https://docs.nvidia.com/deeplearning/tensorrt/latest/inference-library/work-with-quantized-types.html

[13] Z. Ye, L. Chen, R. Lai *et al.*, "FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving," *arXiv preprint arXiv:2501.01005v2*, 2025. [Online]. Available: https://arxiv.org/abs/2501.01005v2

[14] V. J. Reddi, C. Cheng, D. Kanter *et al.*, "MLPerf Inference Benchmark," *arXiv preprint arXiv:1911.02549*, 2019. [Online]. Available: https://arxiv.org/abs/1911.02549

[15] T. Kalibera and R. Jones, "Rigorous benchmarking in reasonable time," in *Proceedings of the International Symposium on Memory Management*, 2013, pp. 63–74. [Online]. Available: https://kar.kent.ac.uk/33611/

[16] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep Reinforcement Learning at the Edge of the Statistical Precipice," *arXiv preprint arXiv:2108.13264*, 2021. [Online]. Available: https://arxiv.org/abs/2108.13264

[17] NVIDIA, "Tegrastats utility," 2025, jetson Linux r38.2 documentation. Accessed: Oct. 19, 2026. [Online]. Available: https://docs.nvidia.com/jetson/archives/r38.2/DeveloperGuide/AT/JetsonLinuxDevelopmentTools/TegrastatsUtility.html

AUTHOR BIOGRAPHIES

Biographies to be completed before submission.